

DATA ENCAPSULATION IN C++

http://www.tutorialspoint.com/cplusplus/cpp_data_encapsulation.htm

Copyright © tutorialspoint.com

```
class Box
{ public:    double getVolume(void)
  {
    return length * breadth * height;
  } private:
  double length;    // Length of a box    double breadth;    //
Breadth of a box    double height;    // Height of a box };
```

```
#include <iostream> using
namespace std;

class Adder{ public:
    // constructor
    Adder(int i = 0)
    {
        total = i;
    }
    // interface to outside world
```

```
void addNum(int number)
{
    total += number;
}
// interface to outside world    int getTotal()
{
    return total;
}; private:
// hidden data from outside world    int total;
};
int main( )
{
    Adder a;

    a.addNum(10);
    a.addNum(20);
    a.addNum(30);

    cout << "Total " << a.getTotal() << endl;    return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Total 60
```

Above class adds numbers together, and returns the sum. The public members **addNum** and **getTotal** are the interfaces to the outside world and a user needs to know them to use the class. The private member **total** is something that is hidden from the outside world, but is needed for the class to operate properly.

Designing Strategy:

Most of us have learned through bitter experience to make class members private by default unless we really need to expose them. That's just good **encapsulation**.

This wisdom is applied most frequently to data members, but it applies equally to all members, including virtual functions.

Loading [MathJax]/jax/output/HTML-CSS/fonts/TeX/fontdata.js